

Destination at Your Command: Intelligent Navigation by Robot Companion

Dhulyatuch Wongsanont¹ Waruna Saowapark² Arckavich Suesatsakulchai³
Panyakorn Singhadoung⁴ Thakorn Jearaborival⁵

Ekamai International School^{1,2}
General Educational Development Program (GED)³
The Demonstration School of Bansomdejchaopraya Rajabhat University⁴
Assumption College⁵

Email : pkofficiate1234@gmail.com

Abstract

By participating in the Thailand Open Robot Competition @Home Education, which aligns with the global RoboCup@Home initiative, we actively contribute to the development of innovative robots capable of handling household tasks. This work ultimately benefits the advancement of domestic robotics and human-robot interaction. In this paper, we comprehensively present the statement of the problem, Problem Solving Approach and discuss the benefits of our approach for mobile robot navigation commanded by human interaction. Our system adeptly detects a person's face, prompts for room destinations, and accurately recognizes voice commands, then commands the robot to navigate autonomously to the specified location.

Keywords: Human, Robot, Interaction, Simulation, Navigation

1. Statement of Problem

The RoboCup@Home education serves as a critical testbed for robotics research, fostering the development of robots capable of handling real-world domestic tasks [1]. Our project takes center stage, leveraging cutting-edge technologies like ROS Noetic [2], OpenCV [3], Google's AI APIs [4], and Turtlebot simulation [5] to push the boundaries of robotic intelligence. We envision a future where robots seamlessly integrate into our homes, understanding our voice commands and navigating environments effortlessly.

Our intelligent robot assistant seamlessly integrates cutting-edge SLAM navigation technology with AI-driven face detection and voice recognition capabilities. This integrated solution offers the following benefits:

- **Improved Time Efficiency:** The robot assists with various tasks, allowing users to focus on other activities simultaneously. Picture assigning chores like laundry retrieval or window inspection while reclaiming valuable time.
- **Enhanced Convenience:** Tedious back-and-forth journeys between rooms are a thing of the past. This robot adeptly manages repetitive tasks, enhancing user comfort and convenience.
- **Nighttime Safety:** In need of assistance during nighttime hours? Simply summon the robot with your voice, mitigating the risk of accidents in dimly lit environments.

2. Project Components

This section provides an overview of the fundamental components integral to our project:

2.1. Face Detection

We utilize OpenCV, an open-source computer vision library, to conduct real-time face detection using a USB camera. OpenCV's comprehensive suite of tools and algorithms enables the accurate identification of faces within live video frames. Upon successful detection, the system initiates subsequent actions as part of the human-robot interaction process.

2.2 The Voice Interaction

Once a person's face is detected, the system engages in dialogue with the user. Utilizing the Google Text-to-Speech API, the system prompts the user to specify the desired destination room audibly. Subsequently, the Google Voice Recognition API transcribes the spoken words into text format, enabling the robot to effectively comprehend the user's specified destination room.

2.3. Turtlebot in House Simulation

We run simulations of the Turtlebot navigating diverse house environments using Gazebo. This simulated environment serves as a crucial tool for testing and refining navigation algorithms without the need for physical hardware. By creating virtual representations of typical household layouts, our system accurately assesses the efficacy and adaptability of navigation strategies in real-world scenarios.

2.4. SLAM Navigation

Simultaneous Localization and Mapping (SLAM) navigation technique enables the Turtlebot to navigate autonomously within its environment by integrating localization and mapping techniques. SLAM enables the Turtlebot to construct a comprehensive representation of its surroundings while determining its position relative to the environment. This capability enhances the robot's capacity to adapt to changes in its surroundings and fulfill user-specified commands. SLAM navigation assumes a critical role in traversing complex indoor environments, delivering users a seamless and efficient interaction experience.

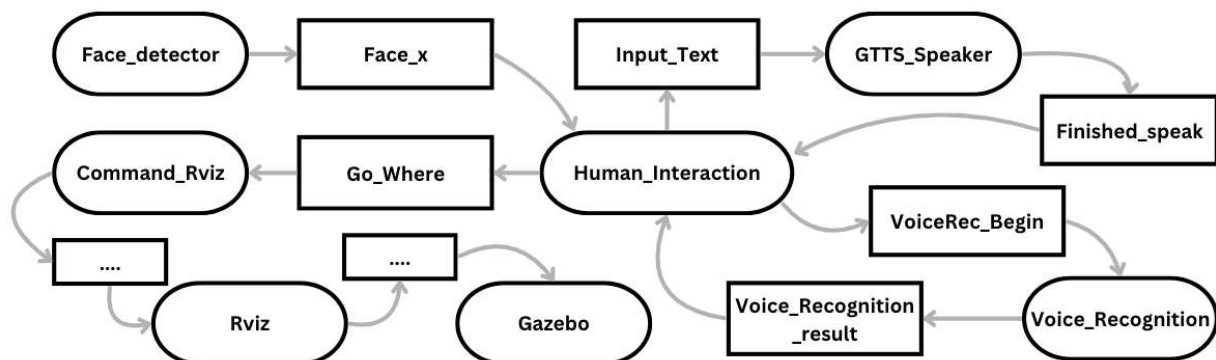


Figure1. Structure of Node and Topic in this project

3. Project Implementation

This section delineates the essential steps involved in configuring and executing our project:

3.1 ROS Noetic Setup on Ubuntu 20.04:

- 1) First we installed ROS Noetic on our Ubuntu 20.04 system.
- 2) Then we set up the necessary environment variables and workspace.

3.2 Library and Dependency Setup:

Installed the following libraries and dependencies for our ROS project:

- 1) Rospy
- 2) Opencv-python
- 3) Google APIs (Google Text-to-Speech and Google Voice Recognition)
- 4) Pyaudio
- 5) Turtlebot 3 simulation including Gazebo and Rviz

3.3 Develop ROS Nodes:

Created the following ROS nodes using Python3:

- Face_Detector Node:

- Utilized OpenCV for real-time face detection using a USB camera.
- Implemented face detection algorithms.
- Triggers the following actions upon successful face detection.

- GTTS_Speaker Node:

- Integrate Google Text-to-Speech API.
- Prompt the user audibly to specify the desired room.

- Voice_Recognition Node:

- Use Google Voice Recognition API.
- Listens and transcribes the user's spoken words into text.

- Human_Interaction Node:

- Handle triggers from Face_Detector and GTTS_Speaker nodes to facilitate interaction between the robot and humans.
- Interact with both Voice_Recognition and Command_Rviz nodes.

- Command_Rviz Node:

- Accepts string from Voice Recognition Node and commands the turtle bot to move to the destination that the user says.

3.4 Using Turtlebot Simulation in Gazebo:

We integrated the Turtlebot simulation within the Gazebo environment. Initially, we set up the Gazebo simulator, followed by importing the Turtlebot model along with a representative house model into the simulation. Subsequently, we configured these models to enable navigation functionalities with the assistance of Rviz. This setup allowed us to simulate the Turtlebot's movement within the virtual house environment, ensuring its ability to navigate through various rooms and obstacles effectively. This integration facilitated the testing and refinement of our navigation algorithms in a controlled virtual environment before deploying them in real-world scenarios.

3.5 Create House Map using Gazebo and Rviz:

We created a house map using Gazebo and Rviz to enhance the navigation capabilities of our Turtlebot. Initially, we conducted simulations where the Turtlebot navigated through diverse house environments, collecting crucial data for mapping purposes. Leveraging the functionalities of Rviz and Gazebo, we directed the Turtlebot to explore the house and gather relevant data. Subsequently, we utilized this data to generate a detailed map of the house layout, essential for facilitating accurate navigation. By saving the generated map, we ensured that the Turtlebot could navigate through the house efficiently, avoiding obstacles and reaching designated destinations with precision. This process significantly contributed to the robustness and reliability of our navigation system within indoor environments.

3.6 Create a Launch File:

We created a launch file to seamlessly launch all our five nodes (Face_Detector, GTTS_Speaker, Voice_Recognition, Human_Interaction, Command_Rviz) at once to make our project more efficient and convenient.

4. Problem Solving Approach

This section elucidates the pivotal technical functionalities of each node within our system:

4.1 Face_Detection Node:

- Function: Continuously captures frames from the USB camera using OpenCV.
- Algorithm: Implements face detection algorithms (Haar Cascade) to identify faces using Opencv within the frames
- Output: Publishes face coordinates to the Human_Interaction node and shows face detection with bounding boxes.
- Trigger: Upon successful detection, triggers the GTTS_Speaker Node.

4.2 GTTS_Speaker Node:

- Function: Integrates with Google Text-to-Speech API.
- Input: Subscribes to the message from Face_Detection Node (confirmation of face detection).
- Action: Converts a predefined message ("Face Detected, where do you want me to go? Speak after you hear the beep sound") into audio using the API and play it through the speakers.
- Output: After the audio has been played, it sends a string "Finished asking" to the Human_Interaction node through the topic "finishes_speak".

4.3 Human_Interaction Node:

- Function: Acts as the center, which subscribes and publishes to many topics. It facilitates interaction between the human and the turtlebot by facilitating interaction with multiple nodes.
- Action: After it has received the string "Finished asking" it will send the string "start_listen" to the Voice_Recognition node through the topic "VoiceRec_Begin".
- Input: It subscribes to the "face_x" and "finishes_speak" topics.
- Output: It publishes to the "text_input" and "VoiceRec_Begin" topics.

4.4 Voice_Recognition Node:

- Function: Utilizes Google Speech Recognition API.
- Input: Records audio from the microphone triggered after it has received the string "start_listen" from the Human_Interaction Node
- Action: Sends recorded audio to the Google API for transcription.
- Output: Publishes the transcribed text (destination room name) as a string to the topic "Go_Where"

4.5 Command_Rviz Node:

- Function: Handles user commands and initiates navigation sequence.
- Input: Subscribes to the topic "Go_where" and receives the user commands as string messages.
- Action:
 - After receiving the string message, it checks whether the string contains any of the predetermined words "start" or "kitchen" or "bedroom".
 - Validates the received destination room name against a predefined map.
 - Triggers the navigation stack with the validated destination.
 - Visualizes the navigation path and turtlebot movement on Rviz.
- Output: Interacts with navigation stack and sends visualization commands to Rviz.

4.6 Navigation Stack (Potential Nodes):

- Move_Base: Transforms the desired goal into a tangible path. Queries the map data (obtained from SLAM) to locate the room's coordinates.
- Turtlebot in Gazebo:
 - Receives velocity commands from Move_Base and executes the received commands, simulating realistic movement within the virtual world, including obstacles and terrain variations.
 - Sensory Feedback: Generates simulated sensor data (e.g., odometry, laser scans) based on its movement, mimicking real-world sensors.
 - Data Sharing: Publishes this sensor data as ROS topics, providing valuable information for other nodes like SLAM for map updates and localization.
- Rviz:
 - Map Exploration: Displays the overall map environment, providing context for the robot's location and path.
 - Monitoring Progress: Offers a clear visual representation of the navigation process, aiding in debugging and performance analysis.
 - Interactive Interface: Allows manual goal setting and manipulation of navigation parameters directly within Rviz for testing and fine-tuning purposes.

5. Test result and Discussion

During testing, the system functioned as demonstrated in the video [6]. Once a face is detected, the system prompts the user to specify where they want the Turtlebot to go. After confirming the command, the Turtlebot moves to the designated location. Throughout our project journey, we gained valuable insights. One significant discovery was related to mapping. If we ventured too far from the house's outer wall during mapping, causing the LIDAR to lose detection, the map became unusable, leading to a discrepancy between the Turtlebot's location in Gazebo and Rviz. To address this, we instructed team members to stay close to the outer wall during mapping, resulting in accurate maps without issues.

Another key finding pertained to project launching. Initially, launching the project required opening multiple terminals, which was cumbersome. However, we simplified this process after learning about launch files. By coding a launch file to combine all five nodes, launching the project became much smoother and less complicated. This improvement streamlined our workflow and made project management more efficient.

6. Conclusion

Our project stands at the forefront of domestic robotics research, offering a blend of benefits that contribute to the advancement of intelligent robots for home environments. Through user-friendly interaction and efficient navigation, we pave the way for enhanced domestic assistance. In summary, our project brings significant advancements in key areas:

6.1 Enhanced Navigation:

We've made navigating through complex indoor environments easy by combining SLAM navigation with powerful algorithms. This adaptability ensures our robot can move efficiently and complete tasks successfully, even when the surroundings change.

6.2 Intuitive Human-Robot Interaction:

Our focus is on making communication with the robot feel natural. With features like face detection, voice recognition, and Google's Text-to-Speech and Voice Recognition APIs, interacting with our robot becomes seamless. This approach fosters trust and acceptance of robots in everyday settings.

6.3 Cost-Effective Development and Testing:

We've optimized our development and testing process by using the Gazebo simulation tool. This means we rely less on physical hardware, saving money and resources while protecting against potential damage. Gazebo also allows us to refine our algorithms in a controlled environment before deploying them in the real world.

7. Acknowledgement

We are deeply grateful to Mr. Noppanut Thongton for his expertise in the preliminary training of ROS principles. His online training materials provided the crucial foundation for our robot's simulation. Additionally, we extend our sincere thanks to Dr. Chacharin Lertyosbordin for sharing his profound knowledge of ROS navigation and AI integration. His guidance was instrumental in equipping our robot companion with intelligent navigation capabilities and an understanding of user commands. Without their dedicated support and expertise, this project would not have reached fruition.

References:

- [1] Iocchi, L., Holz, D., Ruiz-del-Solar, J., Sugiura, K., & Van Der Zant, T. (2015). RoboCup@Home: Analysis and results of evolving competitions for domestic and service robots. *Artificial Intelligence*, 229, 258-281.
- [2] Blubaugh, D. A., Harbour, S. D., Sears, B., & Findler, M. J. (2022). Noetic ROS Further Examined and Explained. In *Intelligent Autonomous Drones with Cognitive Deep Learning: Build AI-Enabled Land Drones with the Raspberry Pi 4* (pp. 417-436). Berkeley, CA: Apress.
- [3] Middi, V. S. R., Thomas, K. J., & Harris, T. A. (2020). Facial keypoint detection using deep learning and computer vision. In *Intelligent Systems Design and Applications: 18th International Conference on Intelligent Systems Design and Applications (ISDA 2018) held in Vellore, India, December 6-8, 2018, Volume 2* (pp. 493-502). Springer International Publishing.
- [4] Choi, J., Gill, H., Ou, S., Song, Y., & Lee, J. (2018, December). Design of voice to text conversion and management program based on Google Cloud Speech API. In *2018 International Conference on Computational Science and Computational Intelligence (CSCI)* (pp. 1452-1453). IEEE.
- [5] Pereira, F. U., de Assis Brasil, P. M., Cuadros, M. A. D. S. L., Cukla, A. R., Junior, P. D., & Gamarra, D. F. T. (2021). Analysis of local trajectory planners for mobile robot with robot operating system. *IEEE Latin America Transactions*, 20(1), 92-99.
- [6] P.K. (2024). Team SEA (High School Level) in Thailand Open Robot Competition @Home Education. Youtube, <https://youtu.be/gehKUz3s5Ys>.